Open in app ↗

Mastering the Art of Pricing Optimization — A Data Science Solution

Unlocking Secrets of Real-World Data Science Solutions for Pricing Optimization in Retail

Rhydham Gupta · Follow
Published in Towards Data Science
16 min read · Aug 28

Listen Share ••• More



Image By Author

Table Of Contents:

1. Overview
2. Elasticity Modeling
3. Optimization

1. Overview

Pricing plays a very crucial role in the world of business. Making a balance between sales and margins is very important for the success of any business. How can we do it in the data science way? In this section, we will build the intuition of an effective data science solution for pricing optimization and then we will go into the details and code of each component.

Note — There are different types of pricing strategies but in this article, we will focus on building the pricing strategy for conventional businesses/established brands with enough data on price change history. Let's look at the basic approach that we are trying to follow before going into details —



Image by Author

We have plotted sales and prices for item 1. In the past 9 months, there have been 2 price changes and clearly, we can see the impact on the sales. When prices were lower sales were higher. Now the question is how to **quantify the impact on sales due to price changes in the past and predict the optimal price for the item in the future.**

An interesting observation from Jan-Apr, the price has been fixed at $5 but we still observe sales fluctuations. It's very normal as in a practical world there are a lot of external factors that impact sales like seasonality, holidays, promotional events, marketing expenditure, etc. **Hence, we don't model the actual sales but the baseline sales which we derive using the different models.**

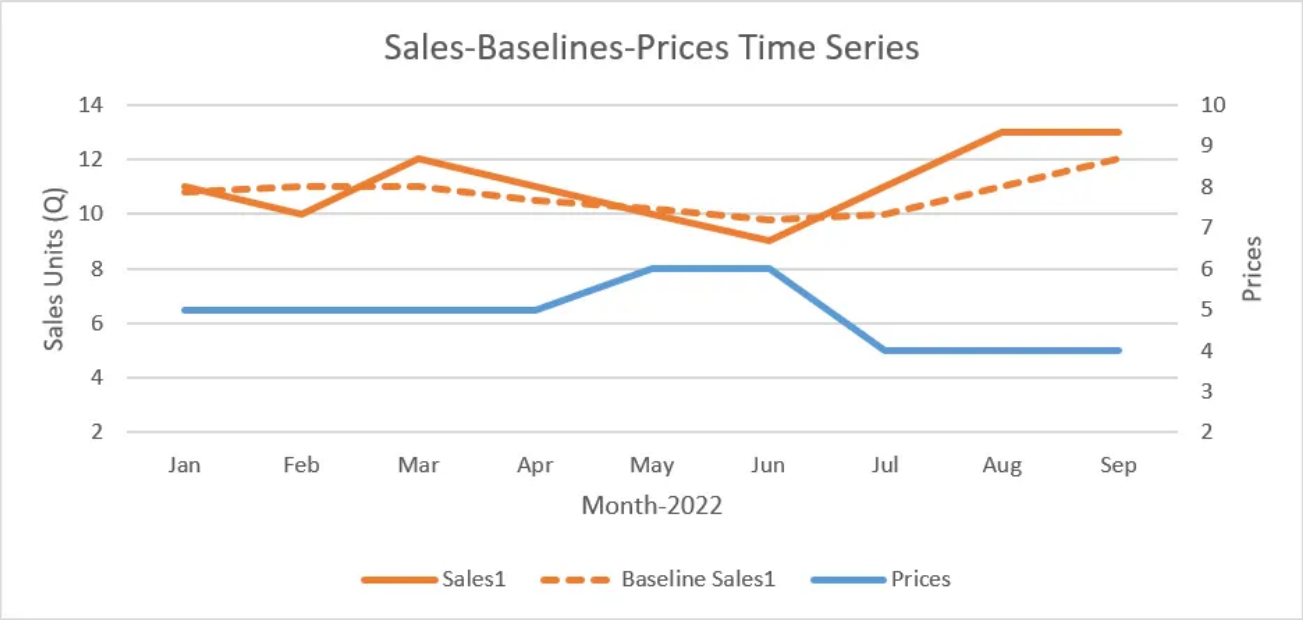


Image by Author

You can observe that we are looking at a smoother trend of sales in the baseline sales series. Is it 100% accurate? Definitely Not! Data science is all about how close we can get to reality. Let's move to the process now —

Suppose, we are hired and asked to provide the prices for thousands of items across stores for the Retailmart group. The prices of the same item can be different across different stores. The company has provided us with data for the past 5 years. What should be our approach to solving it?

Let's understand this with a price meter example. Assume that we have a price meter and we have fixed the minimum and maximum values and the dial can move in between these two extremes. Currently, the dial is pointing toward the current price. Our objective is to stop the dial at a point where we can maximize profits.

Now as we move the dial toward the right (i.e. we are increasing the price) the sales will start to go down and margins will increase but

1. Can we quantify this decrease? yes, we can and it is known as the price elasticity of an item. In simple terms, **Price elasticity** for an item would mean the percentage change in sales for a 1% change in the price.
2. In the real world, sales are often driven by promotional events, holidays, extra discounts, etc. but for optimizing the price we would need to exclude the effect of all these external factors and compute **baseline sales.**

3. Once we quantify the change in sales w.r.t the change in price, **we need the answer, Where do I stop the dial?** For that, we need an objective which in most cases is maximizing the profits. Profits = Sales * Margin so we need to stop at the position where our profits value gets maximum. Mathematically, this is a concept of non-linear optimization where value can move within bounds.

4. Business Rules are important, we have to make sure that the final recommended prices adhere to these rules.

So these are the major steps that we will follow to derive the right price for each item in a given store. Let's look at these steps in a little more detail —

1. Baselines Sales/Base Units

This step is a pre-step for the subsequent steps. As stated, we want to model the impact on sales due to changes in the prices. The ideal scenario is to have sales that are only impacted due to prices but in the practical world, it is never the case —

So we want to simulate the sales for our ideal scenario and we do it using a time series model on the below equation —

> Sales ~ function[Baseline_sales + (promotional effect) + (holidays effect) + (any other effect)]

Please note that sometimes we don't have actual data on the external factors that are impacting the sales. In such a scenario, we can use dummy variables to account for all such factors. A simple example can be if, in a certain month, we see a sudden increase in sales but the price remains constant, we can introduce a simple dummy variable having 1s for that month and 0s for the remaining months.

2. Price Elasticity

Price Elasticity refers to the % change in sales w.r.t the % change in the price of an item for a given store.

As an example, consider two products milk and ABC green tea. Which do you think will have high price elasticity?

Milk, being an essential everyday item with high competition, exhibits high price elasticity. Even a slight change in price can significantly influence sales due to its widespread demand. On the other hand, ABC green tea, which might be available in

a limited number of stores, experiences low price elasticity. A small price change for ABC green tea is unlikely to have a substantial impact on sales due to its niche market presence.

How will we model this?

$$Baseline_Sales \sim function[(price) + trend]$$

The coefficient of the price variable will be used as the price elasticity. Trend variable is used to account for sales increase due to a long-term trend and not necessarily due to price changes. We will discuss more details on computing elasticities in the Price Elasticity section below.

3. Non-linear optimization within bounds

In this step, we will get the answer to where the dial should be stopped.

We first define our objective function — **maximizing the profits**

Then we define the start and end point of the price meter that defines the LB and UB of the price

We have already computed baseline sales and price elasticities which quantify the sales sensitivity to price. We will put all these inputs into our non-linear optimization function and we will get the optimized price.

In very simple words, the algorithm will try different price points within the bounds and check the value of the objective function which in our case is profits. It will return us to the price point where it could get maximum value for our objective function. (In linear optimization, visualize how the gradient descent works). We will discuss more details on computing the optimized prices in the Optimization section below.

4. Business Rules

So can we directly implement the optimized prices in our stores?

No, but what is left now? Adhering to business rules is one of the most important requirements for any business.

But what kind of rules are we talking about in pricing —

1. Ending Number Rules — It is a common practice to price the product at $999 or $995 instead of $1000. There are several psychological reasons for doing so hence we will need to make sure that our final recommended prices adhere to any such rules if these are applicable.
2. Product Gap Rules — Can you sell the one-pack Maggi priced more expensive per unit than the 4-pack Maggi? No, Right. Often if the pack's size increases the per-unit cost should go down or at least stay the same.

So, these are examples of some of the business rules that the business wants to apply. We will do some post-processing steps on the optimized prices to get to the final recommended prices.

Now that you are aware of the overall process, it is time to dive into more details and coding.

2. Elasticity Modeling

In this section, we will understand how we can use this concept to derive optimized prices for 1000s of items across multiple stores. Let's say we have to determine price elasticity for a snack item Yochips selling in a California store for the past 3 years. Let's first see the definition of **price elasticity**:-

> Price elasticity is defined as the percentage change in sales with a 1% change in the price.

Now you may be wondering, which algorithm can I use for computing the price elasticity of an item like Yochips?

Let us look into some details from the economic book on **the constant price elasticity model** and see if we can relate it to some data science algorithm.

The multiplicative form of the demand function will be:-

> $Yi = \alpha * Xi^{\beta}$ (where y will be sales/demand, x will be price and β price elasticity)

Taking the log on both sides

$$\log(Y_i) = \log(\alpha * X_i \hat{} \beta)$$

$$\log(Y_i) = \log(\alpha) + \beta * \log(X_i) \text{ .....Eq(1)}$$

log(α) can be considered as an intercept as β_0

$$\log(Y_i) = \beta_0 + \beta_1 * \log(X_i) \text{ .....Eq(2)}$$

Now taking the differentiation on both sides, we will get

$$\delta Y/Y = \beta_1 * \delta X/X$$

The term on the left-hand side represents the % change in Y which is the % change in sales while the term on the right side represents the %change in price. Now when

%change in price = 1%; then $\delta X/X = 1$

$\delta Y/Y = \beta_1$

This would mean the percentage change in the sales will be **β_1 and that is our elasticity.**

Now if you would have noticed, Equation 2 is a **Regression Equation where log(sales) are regressed against log(price) and the coefficient of log(price) will be our price elasticity.**

Hurray! Now we know that computing elasticity is as simple as training a regression model.

But there is one more catch. The demand function equation has some assumptions and one of the assumptions is that sales are only impacted by price but that is generally never the case in the real world because there are generally multiple factors impacting the sales like promotions, holidays, events, etc. So what is the solution, we need to compute the sales component wherein we can remove the impact of all these additional events.

One thing to clarify is that base sales refer to unit sales and not dollar sales. So in equation 2, we need to regress the price against base units instead of actual unit

sales. Now the question is how can we derive the base units from the actual sales units?

Let's understand it using an example. Below you can see the weekly plot of the time series for sales units and sell prices:-

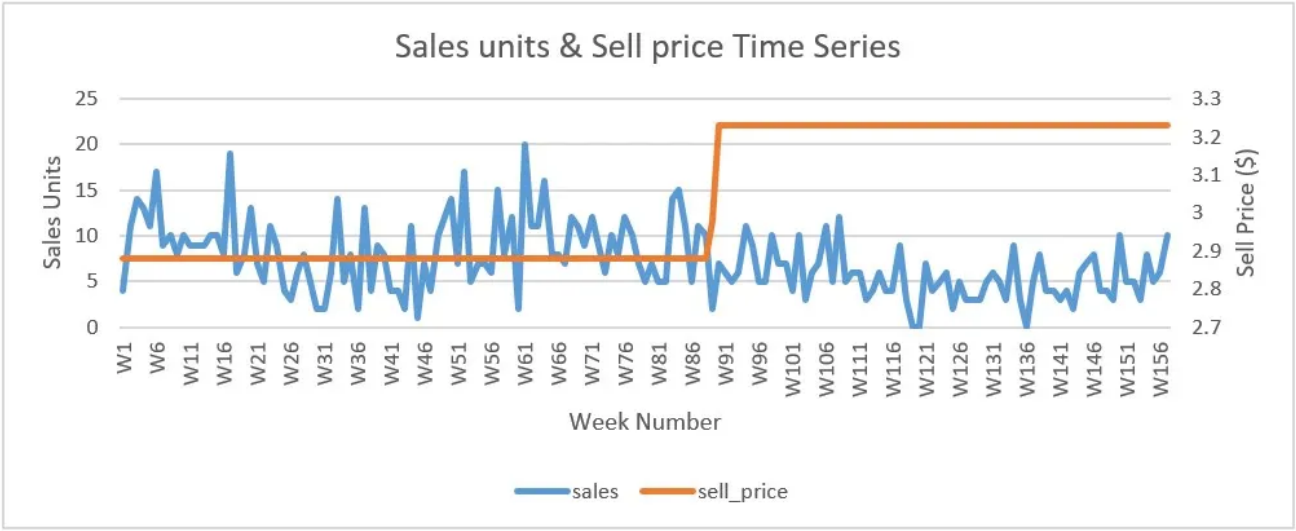


Image by Author

Can you see any pattern in the above plot, it is difficult to tell because there are too many fluctuations in the sales units series. Now these fluctuations can be due to multiple factors like holidays, promotions, events, FIFA World Cup, etc. To isolate the effect of price changes, we need to compute sales that exclude the influence of these additional factors.

Using the prophet model, we can decompose the time series and extract the trend component which represents the base sales. By applying this technique, we separate the long-term impact of price changes from other short-term influences. Let's see what we are talking about:-

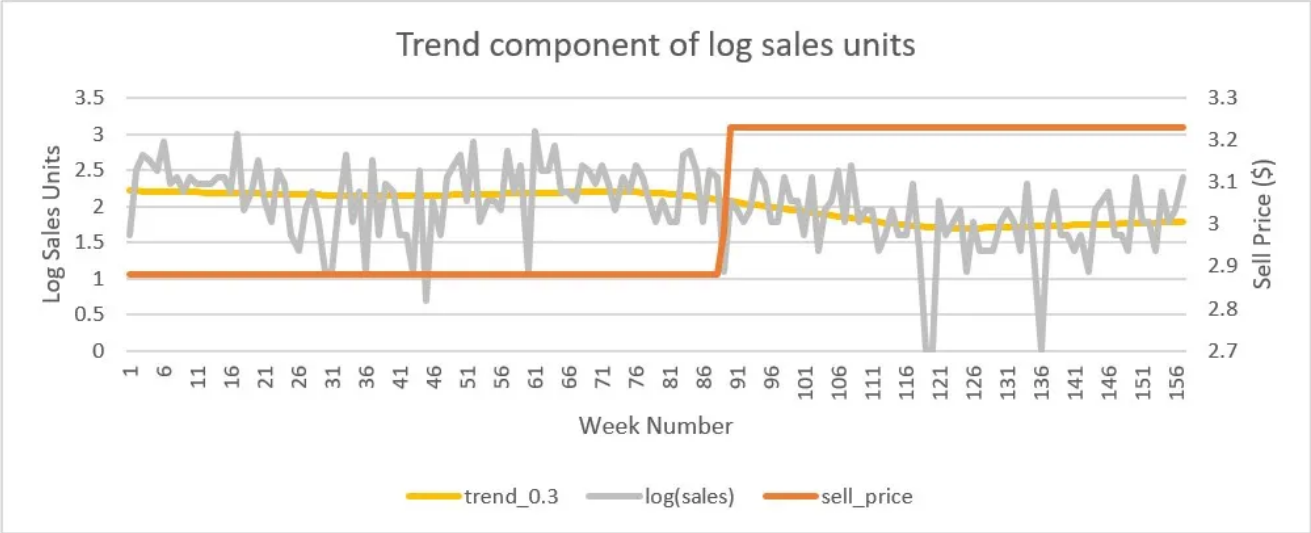


Image by Author

In the above plot, have decomposed the original log sales units (gray) into log base units (yellow line plot)

Here is a code using which you can decompose the time series and fetch the trend component which will become base sales:-

```
# Defining the inputs
timestamp_var = "week_ending_sunday"
baseline_dep_var = "ln_sales"
changepoint_prior_scale_value = 0.3
list_ind_vars_baseline = ['event_type_1_Cultural','event_type_1_National','even
```

```
# Preparing the datasecloset
df_item_store = df_item_store.rename(columns={timestamp_var: 'ds', baseline_dep
df_item_store['ds'] = pd.to_datetime(df_item_store['ds'])

# Initializing and fitting the model
model = Prophet(changepoint_prior_scale= changepoint_prior_scale_value) #Defaul

# Add the regressor variables to the model

for regressor in list_ind_vars_baseline:
    model.add_regressor(regressor)
model.fit(df_item_store)

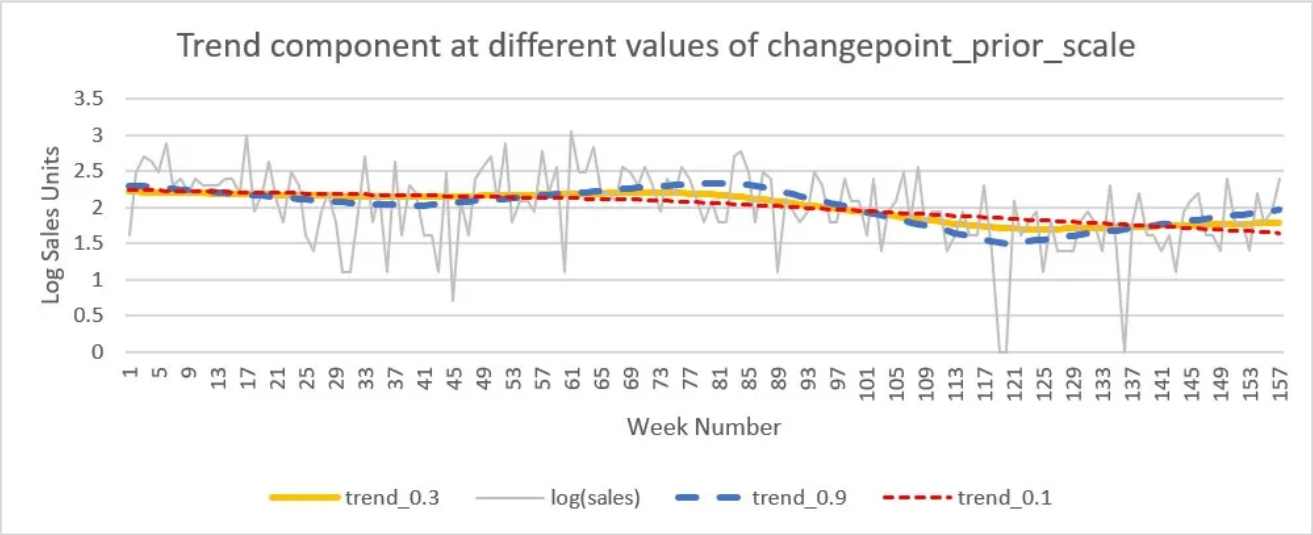
# Since we are only decomposing current time series, we will use same data is f
# Making predictions and extracting the level component
```

```
forecast = model.predict(df_item_store)
level_component = forecast['trend']
```

Below are the inputs that we need to define:

- changepoint_prior_scale_value — This controls the smoothness of the trend. You can read more about it in the prophet model documentation.
- list_ind_vars_baseline — These include all the additional events that have impacted the sales like some festivals, sports events, cultural events, etc.

Here is how changepoint_prior_scale_value impacts the trend. when the value is small then it leads to an almost straight line and when the value is high then the trend is less smooth.



The code is simple, Initially, we rename the “ln_sales” variable as “y” and the “week” variable as “ds” to meet the prerequisites for utilizing the Prophet model. Next, we initialize the Prophet model, specifying the “changepoint_prior_scale” parameter. Subsequently, we incorporate additional event and holiday variables into the model. Finally, we generate forecasts using the same dataset on which we trained the model and extracted the trend component.

Great. We now have the base units series and we can fit a linear regression model between base units (which are already at log scale because we decomposed the log_base_units series) and log(price). Below is the equation:-

$$\log(\text{base units}) = \text{intercept} + \text{elasticity} * \log(\text{price})$$

With the above equation, we can compute the elasticity value. In practice, not all the series are good to model so you may expect some unexpected values of elasticity for various items. Then what is the solution? If we can somehow perform a regression with constraints on the elasticity value. But how can we implement it? Using Optimization function.

For any optimization, below are the basic requirements:-

1. An objective function — This is the equation that we try to minimize/maximize. In our case, it will be the loss function that we use in linear regression MSE (pred-actual => [**intercept** + **elasticity***ln_price — actual]^2)
2. The initial values of the parameter that we are trying to optimize, in our case it is intercept and elasticity. These can be any random values initially.
3. Bounds for the parameters, These are min and max bounds for both intercept & elasticity
4. Optimization algorithm — This is dependent on the library but you can use the defaults and that should give you the right results

Let's now see the code:-

```
# Preparing the matrix to feed into optimization algorithm
x = df_item_store_model
x["intercept"] = 1
x = x[["intercept","ln_sell_price","ln_base_sales"]].values.T
# x_t = x.T

actuals = x[2]
```

```
from scipy.optimize import minimize

# Define the objective function to be minimized
def objective(x0):
    return sum(((x[0]*x0[0] + x[1]*x0[1]) - actuals)**2) # (intercept*1 + elast
```

```
# Define the initial guess
x0 = [1, -1]

# Define the bounds for the variables
bounds = ((None, None), (-3,-0.5))

# Use the SLSQP optimization algorithm to minimize the objective function
result = minimize(objective, x0, bounds=bounds, method='L-BFGS-B')

# Print the optimization result
print(result)

# Saving the price elastitcity of an item in the dataframe
price_elasticity = result.x[1]
df_item_store_model["price_elasticity"] = result.x[1]
```

Notice that we have defined the initial parameter value for intercept as 1 and elasticity as -1. There are no bounds defined for intercept whereas bounds of (-3,-0.5) are defined for elasticity. This is the main reason why we are performing the regression through the optimization function. After running the optimization, we save the optimized parameter value of the price elasticity. Hurray! we have computed the price elasticity!

So our Yochips at the California store have a price elasticity of -1.28.

Let's look at the price elasticity for some other series as well:-

Low Price Elasticity: No significant change in sales with an increase in price. Below is a plot for an item having a price elasticity of -0.5

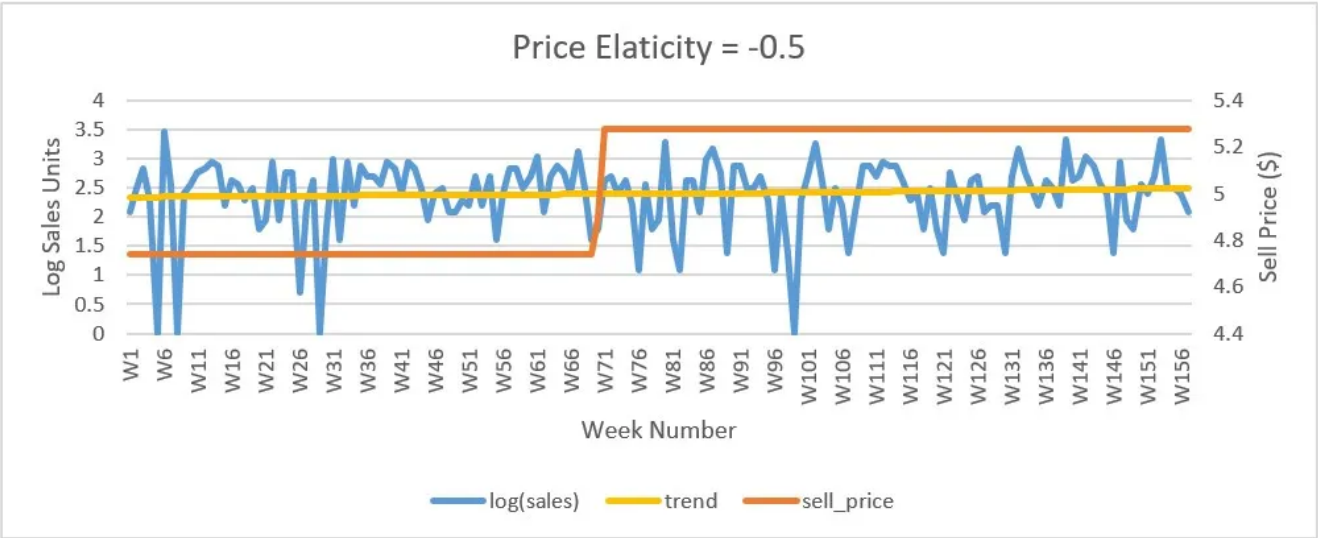


Image by Author

Medium Price Elasticity: There is a moderate drop in sales with an increase in price. Below is a plot for an item having a price elasticity of -1.28

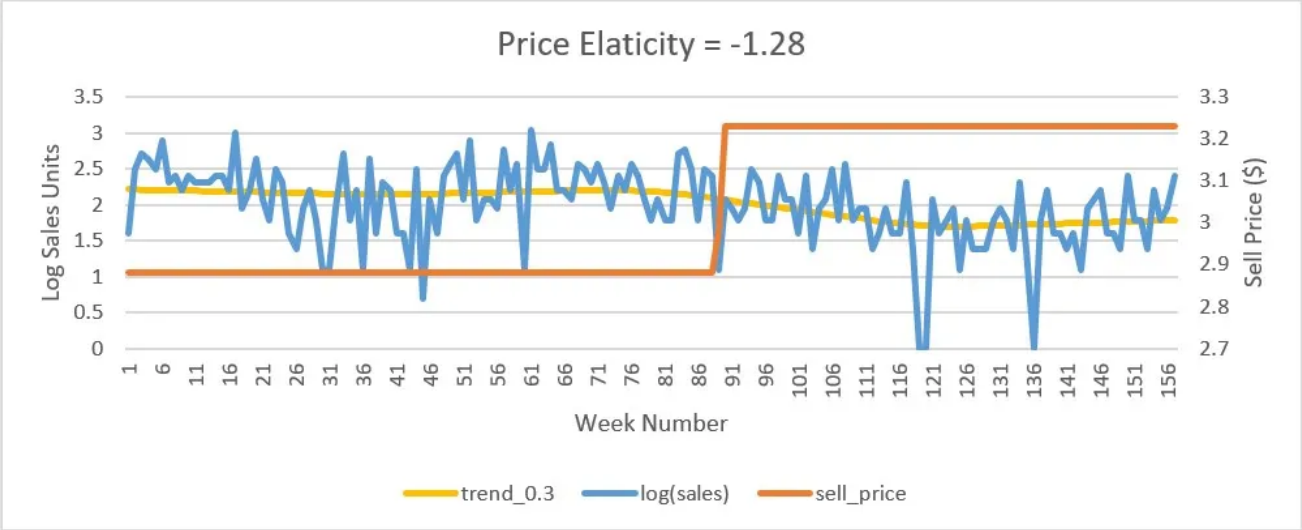


Image by Author

High Price Elasticity: There is a high drop in sales with an increase in price. Below is a plot for an item having a price elasticity of -2.5

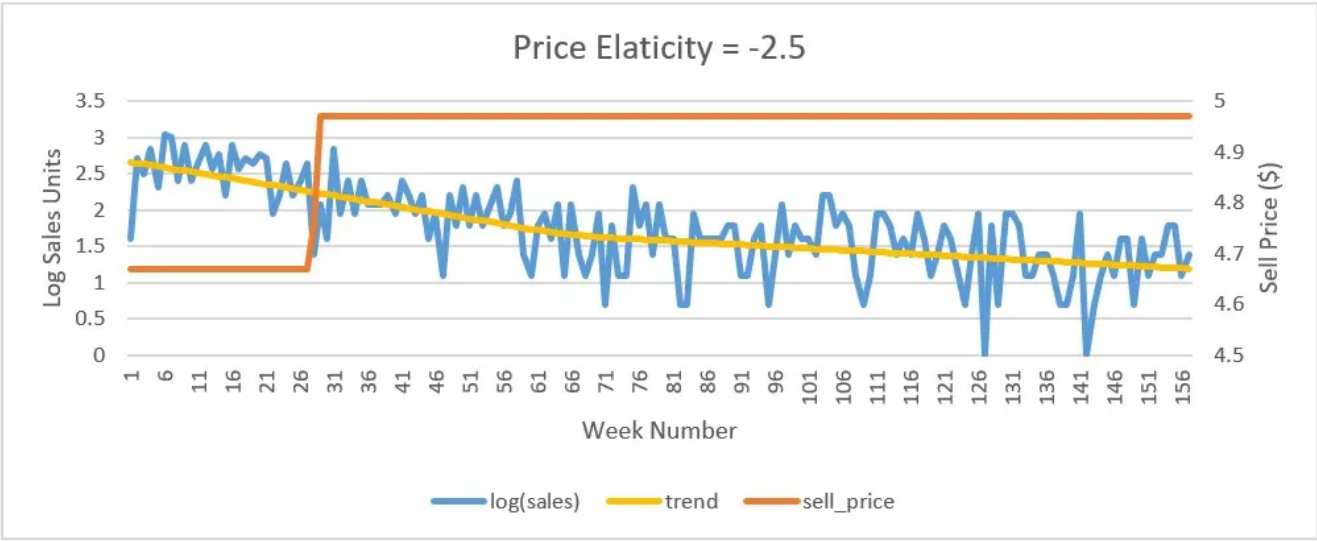


Image by Author

Using the same approach, we can compute the price elasticity for all the items. In the next article, we will explore how we can utilize these elasticity values to determine the optimized prices for each item.

3. Optimization

In the previous section, we had already determined the price elasticity of Yochips for our California store. But that doesn't help the store manager, he wants to know how he should change the price of Yochips to maximize revenue. In this article, we will understand the methodology to optimize prices.

But before that, there are a few questions, we must ask the store manager.

Q: Is there some limit on the minimum and maximum price change that should be considered while optimizing prices?

Based on the discussions with the store manager, we established that the price decrease should not exceed 20% and price increases should also be limited to 20%.

The current price for the Yochips is $3.23 and now we know that the optimized price has to be between $2.58 — $3.876. But how can we derive an optimized price?

But how can we derive an optimized price that maximizes revenue? Let's do some maths:-

> *Optimized Revenue = Total units sold * (**Optimized Price**)*

We need to optimize the price so that we can maximize revenue. But the total units sold will also change with a change in price. Let's re-write the above equation and we can call the total units sold at an optimized price as optimized units:-

> *Optimized Revenue = Optimized units* (**Optimized Price**)……(eq1)*

We already know that —

> *Elasticity = %change in the units sold/ %change in the price*

Therefore:-

> *Optimized units = Base units + change in units at an optimized price*

Here, Base units refer to total unit sales at the current price which is $2.58

> *Optimized units = (Base units + (Base units * price elasticity * (% change in the optimized price vs regular price) ……. (eq2)*

Let's impute the eq2 in eq1

> *Optimized Revenue = (Base units + (Base units * price elasticity * (% **change in the optimized price vs regular price**) * (**Optimized Price**) …….. (eq3)*
>
> *Optimized Revenue = (Base units + (Base units * price elasticity * [(**Optimized Price** — Current Price)/ Current Price]* (**Optimized Price**)……..eq(4)*

Below are the key parameters in the optimization equation (eq4):

Base units = Average unit sales at the current price.

Price elasticity = computed value for the item's price elasticity

Current price = Latest selling price

Great! In our equation, apart from the optimized price, we have data for all other variables. So which algorithm can we use to compute an optimized price that maximizes revenue? **We can simply use the optimization algorithm.**

What are the critical components needed for optimization:-

1. **The objective function that needs to be minimized/maximized**:- We have already defined the objective function which is maximizing the optimized revenue as defined in eq(4).
2. **Bounds**: As defined by the store manager, we need the optimized price to not change by more than 20%. So the Lower Bound = Current Price (1–0.2) & Upper Bound = Current Price (1+0.2)
3. **Optimization Algorithm**: We will use Scipy.optimize library from Python to implement the optimization.

Let's look at the code:-

```
# Taking latest 6 weeks average of the base sales
#-------------------------------------------------

# Ranking the date colume
df_item_store_optimization["rank"] = df_item_store_optimization["ds"].rank(asce

# Subset latest 6 weeks of data
base_sales_df = df_item_store_optimization.loc[df_item_store_optimization["rank
```

```
df_item_store_optimization_input.rename(columns = {"base_sales":"base_units"},

# Deriving the min and max bound for the sell_price
#-------------------------------------------------

# Creating UB and LB as with the range of 20%
df_item_store_optimization_input["LB_price"] = df_item_store_optimization_input
df_item_store_optimization_input["UB_price"] = df_item_store_optimization_input
```

The above code helps us in data prep for optimization. Firstly, we are computing the base units as the average of base_sales (trend component of the decomposed series) for the latest 6 weeks. We have already discussed the methodology to compute the base_sales in the above section.

Next, we are defining the LB_price & UB_price by decreasing and increasing 20% from the current sell price respectively.

Let's define the code to carry out optimization.

```
from scipy.optimize import minimize

# Define the objective function to be minimized
def objective(opti_price):

    df_item_store_optimization_input["opti_price"] = opti_price
    df_item_store_optimization_input["optimized_units"] = df_item_store_optimiz



    df_item_store_optimization_input["optimized_revenue"] = df_item_store_optim

    return -sum(df_item_store_optimization_input["optimized_revenue"])

# Define the initial guess
opti_price = df_item_store_optimization_input["sell_price"][0]

# Define the bounds for the variables
bounds = ((df_item_store_optimization_input["LB_price"][0], df_item_store_optim

# # Use the optimization algorithm to minimize the objective function
result = minimize(objective, opti_price, bounds=bounds)
```

```
    # Print the optimization result
    print(result)
```

The above code will give us the optimized price. Can you guess that in the objective function why are we defining negative optimized revenue? What is -(-1), it is 1. We are minimizing the objective function and using the negative sign for the optimized revenue will lead to maximizing the optimized revenue.

Moreover, we can initialize the opti_price variable with any random value, just to facilitate fast convergence, we are initializing it with the current sell_price. In the bounds, we are defining the LB & UB that we have created in the above code.

Hurray! We have found out the optimized price for Yochips and we are ready to propose it to the California store manager.

```
print("Inital price ",df_item_store_optimization_input["sell_price"][0])
print("Optimized price ",result.x)
print("Recommended price change",result.x[0] - df_item_store_optimization_input["sell_price"][0])
```

```
Inital price  3.23
Optimized price  [2.90700006]
Recommended price change -0.3229999542236328
```

Image by Author

Our recommendation will be to drop the Yochips prices by 10.2% to $2.9. This will result in maximum revenue.

This is the last step in the price optimization approach and the overall approach is so powerful that it can help us to return optimized prices for each item at every store.

One of the limitations of the above approach is for the items where we don't have sufficient price change history. In that scenario, we use other techniques but if that fraction of items is less, then the average price elasticity at category level can be used for such items.

Hope you enjoyed this article! Now you know how to do Pricing Optimization.



Image by Author (Fonts made from onlinewebfonts)

Pricing Analytics | Data Science | Retail | Optimization | Analytixtime



Follow

Written by Rhydham Gupta

562 Followers · Writer for Towards Data Science

I am a Data Scientist, I believe that observing and decoding data is an art. Same Data, Different Eyes Different Stories